

Carbon Commit

A High-Fidelity PL/SQL Framework for Institutional Carbon Accounting

UCS310 – Database Management Systems

Department of Computer Engineering

Submitted by:

Manroop kaur-1024170049

Avneet kaur-1024170050

Kunwarpreet Singh-1024170439

BE Second Year, 2Q12

Submitted to:

Dr. Tanya Garg



1. Introduction

The application domain for **Campus Commit** is **Institutional Environmental Management and Sustainability Auditing**. As Thapar Institute aggressively pursues "Green Campus" certifications and carbon-neutral status, the ecological data generated—ranging from high-voltage laboratory energy usage to organic waste from multiple hostel mess halls—has increased in both volume and complexity, surpassing the capacity of manual oversight.

Motivation for Selection

The motivation for this project stems from the urgent requirement to precisely quantify the carbon footprint (CO₂e) of diverse campus activities. In a large-scale institution, emissions are not monolithic; they are decentralized across departments, student residences, and transport hubs. Without a centralized digital ledger, identifying "emission hotspots" or tracking the success of sustainability initiatives like solar grid offsets becomes statistically unreliable.

DBMS vs. File-Based Systems

Utilizing a **Database Management System (DBMS)** is technologically superior to a traditional file-based system (such as spreadsheets) for several critical reasons:

- **Structured Data Storage:** Unlike flat files, a DBMS allows for a normalized relational schema that eliminates data redundancy and ensures that environmental constants are stored in a single, verifiable location.
- **Concurrency Control:** A campus-wide system requires simultaneous access; a DBMS manages multi-user access across different departments, preventing "Lost Updates" when multiple hostel wardens or lab technicians input data at the same time.
- **Data Independence:** The system ensures that the complex backend logic for emission calculations (stored in PL/SQL) remains consistent and isolated. This means the "math" behind the sustainability audit is protected even if the user interface or frontend reporting tool changes.
- **Data Integrity & Security:** Through the use of schema-level constraints (Primary Keys, Foreign Keys, and Check Constraints), the system guarantees that only valid, non-negative, and non-duplicate environmental logs are processed.

2. Problem Statement

Thapar Institute of Engineering & Technology (TIET), Patiala, is a large residential campus comprising multiple academic departments, hostels, administrative buildings, laboratories, mess facilities, and transportation systems. These units consume significant amounts of electricity, water, fuel, paper, and other resources on a daily basis.

Currently, sustainability-related data such as energy consumption, waste generation, and transportation usage is maintained separately by different departments and facility units. There is no centralized system to track the overall carbon footprint of the campus.

The proposed project, **Carbon Commit: Carbon Emissions Monitoring & Management System**, aims to develop a database-driven solution to monitor, store, and analyze environmental impact data across TIET. The system will calculate carbon emissions using standardized emission factors and provide actionable insights to help the institute move toward green campus initiatives and sustainable operations.

Limitations of the existing/manual system:

The current system of record-keeping at the institutional level relies largely on manual registers, spreadsheets, or isolated digital files maintained by individual departments. This approach has several limitations:

- Lack of centralized data management across campus units.
- Inconsistent data formats across departments.
- Limited ability to analyze trends over time.
- No real time monitoring or decision-support system.

These issues make it difficult for the institute to implement effective sustainability policies or qualify for environmental certifications.

The primary challenge facing the institution is that current sustainability tracking is decentralized, relying on fragmented, departmental spreadsheets that lack a unified architecture. This reliance on a "file-based" approach creates significant roadblocks to achieving a truly "Green Campus" status.

2.1 Manual Limitations and Update Anomalies

In the existing system, critical data like **Carbon Emission Factors** (e.g., the

specific grid intensity of the power supplied to Patiala) are hard-coded into separate departmental files.

- **Update Anomalies:** If the emission factor changes—due to an increase in renewable energy sourcing by the Punjab State Power Corporation—administrators must manually update dozens of independent spreadsheets across various hostels and departments.
- **Inconsistency:** Failure to update even a single file leads to "Inconsistent Data," where two departments might report vastly different carbon footprints for the same amount of energy consumed.

2.2 Lack of Data Integrity and Validation

Manual systems offer no robust mechanism to enforce business rules at the point of entry.

- **Invalid Entries:** There are currently no constraints to prevent a user from entering physically impossible values, such as negative electricity units or negative food waste kilograms.
- **Redundancy and Duplication:** Without a central authority to check for uniqueness, duplicate logs for the same building or the same date often occur, leading to "Double Counting" and skewed sustainability reports.

2.3 Database Scope and Proposed Solution

The **Campus Commit** project is designed to resolve these systemic issues by centralizing all institutional activity into a single, **3NF normalized relational schema**.

- **Centralization:** By moving constants to a dedicated **EMISSION_REF** table, any change in an emission factor is updated in exactly one row, instantly reflecting across all historical and future logs.
- **Automation via PL/SQL:** The project uses **triggers** to enforce real-time data validation and **stored procedures** to automate the generation of campus-wide audits, replacing hours of manual calculation with milliseconds of database processing.
- **Integrity Enforcement:** The scope includes implementing Primary Key, Foreign Key, and CHECK constraints to ensure that every byte of data stored is accurate, unique, and audit-ready.

3. Objectives of the Project

The primary goal of this project is to develop a robust, automated environmental accounting system that replaces manual tracking with an intelligent database framework. The specific objectives are:

- To design a comprehensive database using E-R data modeling that accurately represents the complex hierarchy of a college campus, including departments, hostels, laboratory facilities, and their respective emission activities.
- To convert the conceptual ER model into a set of relational tables with strictly defined Primary Key and Foreign Key relationships to ensure referential integrity across all institutional data.
- To apply advanced normalization techniques (up to 3NF/BCNF) to eliminate data redundancy, thereby preventing update anomalies and ensuring that emission constants are stored in a single, verifiable location.
- To implement a secure backend using SQL (DDL/DML) for schema creation and efficient manipulation of high-volume consumption data.
- To develop intelligent PL/SQL components, including Stored Procedures and Functions, to automate the conversion of raw energy/waste metrics into carbon equivalents (CO₂e) and use Triggers to provide real-time alerts for sustainability threshold violations.
- To ensure data consistency and reliability by implementing ACID-compliant transactions for carbon credit transfers and utilizing Check Constraints to prevent the entry of invalid or negative environmental data.
- To integrate the backend logic with a responsive frontend interface (such as Oracle APEX) that allows different campus stakeholders to perform data entry and view analytical dashboards without needing direct access to the SQL command line.

4. Scope of the Project

The scope of **Campus Commit** defines the functional boundaries of the system, ensuring it remains a focused and high-performance database solution for institutional use.

- **User Roles & Access Control:**
 - **Admin:** Responsible for system configuration, managing user credentials, and updating the global `EMISSION_REF` table constants.
 - **Staff (Data Entry):** Authorized to input daily consumption logs for specific buildings, such as Mess Managers entering waste data or Hostel Wardens logging meter readings.
 - **Management (Deans/Department Heads):** Granted "Read-Only" access to high-level analytical dashboards and monthly audit reports to aid in policy-making.
- **Core Modules:**
 - **Resource Monitoring Module:** Focused on the backend logging of electricity (kWh), water (liters), and generator fuel consumption.
 - **Sustainability & Offset Module:** Tracks positive environmental contributions, such as units generated by campus solar panels and the carbon sequestration of campus tree plantation drives.
 - **Waste & Mess Analytics:** Dedicated to tracking the biomass of food waste and identifying consumption patterns across different hostels.
- **Functional Boundaries:** The system is localized to the Thapar Institute campus and does not account for external student commutes or third-party vendor supply chains outside of campus operations.

5. Proposed System Description

The **Campus Commit** system is an integrated, backend-heavy database solution designed to serve as the definitive ledger for all environmental activities within the institution. Unlike traditional applications where the logic resides in the user interface, this system embeds its "intelligence" directly into the **Oracle/MySQL backend** using **PL/SQL**, ensuring that all sustainability rules are enforced regardless of how the data is accessed.

5.1 Systematic Workflow & Automation

The operational workflow is designed for maximum automation and minimal human intervention:

- **Automated Computation:** When a staff member inputs raw consumption data (e.g., total units of electricity for Hostel J), the system triggers a **Stored PL/SQL Function**. This function performs a real-time join with the **EMISSION_REF** table to fetch the most recent carbon constant and returns the CO2 value instantly.
- **Intelligent Auditing:** A **"Before-Insert" Trigger** intercepts the data entry to compare the new value against the building's **Baseline_Quota**. If the entry indicates a violation of sustainability targets, the system automatically populates the **ALERTS** table, creating a proactive notification system for management.
- **Batch Processing:** Instead of manual report compilation, the system utilizes **Stored Procedures** combined with **Cursors** to iterate through all departments. This allows for the generation of campus-wide monthly audits with a single command, converting raw data into actionable insights.

5.2 Quantifiable Efficiency Gains

The shift from a file-based or manual system to a centralized DBMS provides significant performance improvements:

- **Reduction in Audit Time:** By shifting calculation and aggregation logic to the database layer, the time required to compile institutional environmental reports is reduced from **days to seconds**.
- **Zero Calculation Drift:** Centralizing constants in the **EMISSION_REF** table ensures that every department uses the exact same multipliers,

eliminating the common "spreadsheet errors" where different files contain outdated formulas.

- **Resource Optimization:** Automated exception handling ensures that invalid data (e.g., negative consumption or non-existent building IDs) is rejected at the entry point, saving hours of manual data cleaning during the audit phase.

5.3 Data Integrity & The "Single Version of Truth"

A core feature of **Campus Commit** is its ability to maintain a reliable and consistent history of the campus environment:

- **Strict Relational Constraints:** Through the use of **Primary Keys** and **Unique Constraints**, the system prevents the recording of duplicate logs for the same building on the same date, ensuring data is never over-counted.
- **Referential Integrity:** By using **Foreign Keys**, the system ensures that every activity log is linked to a valid department in the `DEPT_MASTER` table, preventing "orphaned" data that can skew total campus averages.
- **Concurrency Control:** The system uses row-level locking and **ACID-compliant transactions** to allow multiple staff members to update mess or hostel logs simultaneously without risking "Lost Updates" or inconsistent partial data.

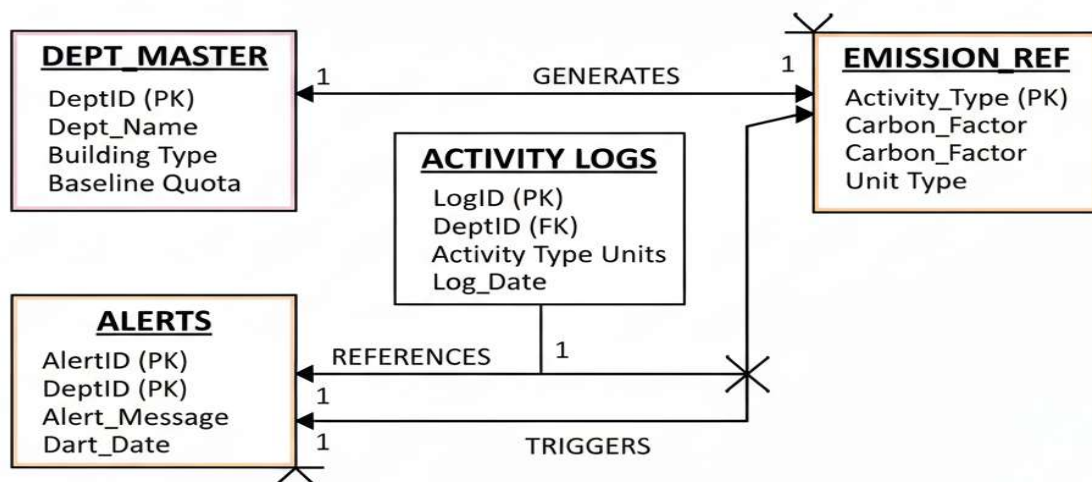
6. Database Design

6.1 Entity-Relationship (ER) Diagram

The ER diagram captures the complex relationships between campus infrastructure and environmental data.

- **Entities Identified:** DEPARTMENTS, ACTIVITY_LOGS, EMISSION_REF, USERS, and ALERTS.
- **Relationships:** A "One-to-Many" relationship exists between DEPARTMENTS and ACTIVITY_LOGS, signifying that one building can generate thousands of data points over time.

Campus Commit Database - Detailed ERD



6.2 Relational Schema & Data Dictionary

The following table structures represent the 3NF implementation of the database.

1. Table: DEPT_MASTER

Stores the static details of campus buildings and their emission limits.

Column Name	Data Type	Constraints	Description
DeptID	NUMBER(5)	PRIMARY KEY	Unique ID for the building
Dept_Name	VARCHAR2(50)	NOT NULL	Name of the hostel/department
Bldg_Type	VARCHAR2(20)	CHECK	Type: 'Academic', 'Hostel', 'Mess'
Baseline_Quota	NUMBER(10,2)	DEFAULT 500	Monthly CO2 limit in KG

2. Table: EMISSION_REF

A reference table for activity constants to prevent data redundancy.

Column Name	Data Type	Constraints	Description
Activity_Type	VARCHAR2(20)	PRIMARY KEY	E.g., 'Electricity', 'Waste'

Carbon_Factor	NUMBER(8,4)	NOT NULL	CO2 multiplier per unit
Unit_Type	VARCHAR2(10)	NOT NULL	E.g., 'kWh', 'kg', 'Liters'

3. Table: ACTIVITY_LOGS

The central transaction table recording daily campus consumption.

Column Name	Data Type	Constraints	Description
LogID	NUMBER(10)	PRIMARY KEY	Unique transaction ID
DeptID	NUMBER(5)	FOREIGN KEY	References DEPT_MASTER(DeptID)
Activity_Type	VARCHAR2(20)	FOREIGN KEY	References EMISSION_REF(Activity_Type)
Units	NUMBER(10,2)	CHECK (Units > 0)	Raw consumption value
Log_Date	DATE	DEFAULT SYSDATE	Date and time of logging

4. Table: ALERTS

Stores automated notifications generated by triggers.

Column Name	Data Type	Constraints	Description
AlertID	NUMBER(10)	PRIMARY KEY	Unique notification ID
DeptID	NUMBER(5)	FOREIGN KEY	References DEPT_MASTER(DeptID)
Alert_Message	VARCHAR2(200)	NOT NULL	Details of the limit violation
Alert_Date	DATE	NOT NULL	When the trigger was fired

Relationship Mapping

- **DEPT_MASTER** → **ACTIVITY_LOGS**: 1:N relationship. A department can have multiple logs, but each log is linked to one DeptID.
- **EMISSION_REF** → **ACTIVITY_LOGS**: 1:N relationship. One emission constant (like Electricity) is used across many log entries.
- **DEPT_MASTER** → **ALERTS**: 1:N relationship. A department can trigger multiple alerts for various violations.

7. Normalization

To ensure the **Campus Commit** database is lean, efficient, and free of anomalies, a strict normalization process was followed up to the **3rd Normal Form (3NF)**. This process ensures that the database avoids redundancy, leading to faster query execution and improved data integrity.

7.1 Identification of Functional Dependencies

Before normalization, we identified the following functional dependencies within our initial unnormalized activity data:

- **LogID** → {**DeptID**, **Activity_Type**, **Units**, **Log_Date**}: The unique identifier of a log determines all basic transaction details.
- **Activity_Type** → {**Carbon_Factor**, **Unit_Type**}: Crucially, the emission multiplier and unit description depend solely on the type of activity, not the specific transaction ID.
- **DeptID** → {**Dept_Name**, **Building_Type**, **Baseline_Quota**}: Building details depend solely on the Department ID.

7.2 The Normalization Process

We followed a step-by-step decomposition to refine the table structures:

First Normal Form (1NF)

- **Action:** We ensured that all table attributes contain only **atomic (indivisible) values**.
- **Result:** Each record is unique, and multi-valued attributes (such as listing multiple "Activities" in one department row) have been removed, ensuring every intersection of row and column contains a single data value.

Second Normal Form (2NF)

- **Action:** We ensured the table is in 1NF and that **all non-key attributes are fully functionally dependent** on the primary key.
- **Result:** We eliminated **partial dependencies**. For example, the `Dept_Name` was removed from the `ACTIVITY_LOGS` table because it depended only on the `DeptID` (part of the logic) rather than the `LogID`. This data was moved to a separate `DEPT_MASTER` table.

Third Normal Form (3NF)

- **Action:** We removed **transitive dependencies** where a non-key attribute depends on another non-key attribute.
- **Result:** In the **ACTIVITY_LOGS** table, the **Carbon_Factor** depended on the **Activity_Type**, which in turn depended on the **LogID**. To resolve this, we moved all emission constants into a separate **EMISSION_REF** table.

7.3 Final Normalized Structure & Advantages

The final structure consists of specialized, linked tables:

1. **DEPT_MASTER:** Stores static building information.
2. **ACTIVITY_LOGS:** Stores purely transactional data.
3. **EMISSION_REF:** Stores reference constants for environmental calculations.

Key Advantage: This 3NF structure prevents **Update Anomalies**. If the carbon factor for "Electricity" changes due to new grid intensity data, the change is made in **one single row** of the **EMISSION_REF** table. Without normalization, we would have been forced to update thousands of historical records in the **ACTIVITY_LOGS** table, leading to potential data inconsistency and system lag.

8. Database Implementation

8.1 SQL Implementation

The system is built using standard SQL commands to manage data and structure.

- **DDL (Data Definition Language):** Using `CREATE TABLE` with integrated constraints like `CHECK (Units > 0)` and `NOT NULL` to ensure data quality from the start.
- **DML (Data Manipulation Language):** Efficient `INSERT` and `UPDATE` operations for logging daily consumption.
- **Advanced Queries:**
 - **Joins:** Combining `ACTIVITY_LOGS` with `DEPT_MASTER` to find the total footprint of "Hostel J".
 - **Aggregations:** Using `SUM()` and `GROUP BY` to calculate the annual campus carbon total.
 - **Views:** A "Sustainability Leaderboard" view that ranks departments by their reduction in waste.

8.2 PL/SQL Components

This section represents the "intelligence" of **Campus Commit**.

- **Stored Procedures:** `Generate_Audit_Report` – Uses a **Cursor** to iterate through all campus departments and generate a summary report into a permanent `AUDIT_HISTORY` table.
- **Triggers:** `Check_Quota_Trigger` – A "Before-Insert" trigger that checks if the new log will exceed the department's monthly `Baseline_Quota` and logs a warning in the `ALERTS` table.
- **Functions:** `Get_CO2e` – A reusable logic block that performs the multiplication of units by the carbon factor, ensuring uniform calculations across the system.
- **Exception Handling:** Implemented within procedures to handle errors such as "Divide by Zero" during average calculations or "No Data Found" when querying new departments.

9. Transaction Management & Concurrency

This section addresses how the system maintains data reliability and handles simultaneous access, ensuring that the "Sustainability Ledger" remains accurate even under heavy load.

- **Atomicity (All-or-Nothing):** By utilizing `COMMIT`, `ROLLBACK`, and `SAVEPOINT` commands, the system ensures that complex operations—such as transferring "Carbon Credits" from a high-performing hostel to a department in deficit—are treated as a single unit of work. If any part of the transfer fails, the entire transaction is rolled back to maintain the original state.
- **Consistency:** The system enforces predefined business rules through database constraints. For instance, a transaction will be rejected if it results in a negative value in the `Baseline_Quota` or violates referential integrity between the `Activity_Logs` and `Dept_Master`.
- **Isolation & Concurrency Control:** To handle multiple department heads entering meter readings simultaneously, the system employs Row-Level Locking. This prevents "Lost Updates," where two users might attempt to update the same energy log at once, ensuring each update is isolated until completion.
- **Durability:** Once a sustainability audit is finalized and "committed," the data is permanently written to the disk, ensuring that campus environmental records are protected against system failures or power outages.

10. Tools & Technologies Used

While the **UCS310** curriculum emphasizes backend implementation, a functional frontend is essential for users like hostel wardens and deans to interact with the **Campus Commit** database. To maintain the professional, industry-standard focus of your project, the frontend should prioritize clean data entry forms and analytical dashboards.

Frontend & Application Layer:

While the core "intelligence" resides in the **Oracle 21c** backend, the following technologies will be used to create the user interface:

- **Oracle APEX (Application Express):** As you are already using Oracle 21c, **Oracle APEX** is the most professional choice. It is a low-code development platform that runs natively within the Oracle Database, allowing you to build responsive, data-driven web applications directly on top of your **SQL and PL/SQL** logic.
- **Web Technologies (HTML5, CSS3, JavaScript):** For building custom dashboard components, such as real-time "Green Leaderboards" or emission gauges that visualize the data retrieved from your **SQL Views**.
- **Data Visualization Libraries (Chart.js / D3.js):** Integrated into the frontend to convert complex **Aggregate Function** results (like monthly CO2 trends) into intuitive bar charts and pie graphs for campus administrators.

Frontend Features & User Modules

The frontend will be divided into specific modules based on the system's **Scope**:

Module	Primary User	Key Frontend Functionality
Data Entry Portal	Staff/Hostel Wardens	Forms with real-time validation for logging electricity units and mess waste.

Executive Dashboard	Admin/Deans	Visual summaries of campus-wide emissions and "High Emission Alert" notifications triggered by the backend.
Student Insights	Users (Students)	A "Personal Footprint" tracker where students can view the impact of their hostel or department.

Integration with Backend Logic

The frontend will not perform any calculations; instead, it will act as a bridge to your **PL/SQL components**:

- **Triggering Actions:** Clicking a "Generate Report" button on the UI will execute a **Stored Procedure** using a **Cursor** to compile data.
- **Displaying Results:** The UI will query **SQL Views** to show pre-calculated sustainability scores, ensuring the application remains fast and responsive.
- **Error Handling:** Any **Exceptions** caught by the PL/SQL blocks (like "Duplicate Entry") will be passed to the frontend as user-friendly error messages.

11. Expected Outcomes

The successful implementation of **Campus Commit** is expected to yield the following measurable improvements over traditional manual systems:

- **Normalized Architecture (3NF):**
 - The creation of a fully normalized database structure ensures that each data element is stored in exactly one location, successfully eliminating data redundancy.
 - This architecture minimizes storage overhead and effectively prevents update anomalies, such as inconsistent emission factors across different records.
- **Optimized Data Retrieval:**
 - By utilizing indexed **SQL Views** and multi-table **Joins**, the system allows for the high-speed generation of "Green Leaderboards" and footprint summaries in milliseconds.
 - This replaces the slow, manual cross-referencing required in file-based systems, enabling real-time sustainability tracking for campus administrators.
- **End-to-End Process Automation:**
 - The project replaces manual sustainability auditing with **PL/SQL triggers** and **stored procedures** that automatically calculate CO2 equivalents at the point of data entry.
 - Automated logic will instantly flag high-emission events, ensuring that institutional baseline violations are recorded without human intervention.
- **Enhanced Data Integrity and Audit Readiness:**
 - The system establishes a robust, timestamped audit trail where every emission entry is verified against institutional **check constraints** and **foreign key** relationships.
 - This ensures **Data Consistency** and provides a "single version of truth" for international campus sustainability certifications and "Green Campus" ratings.
- **Transactional Reliability (ACID Compliance):**
 - Through the use of **COMMIT** and **ROLLBACK** commands, the system guarantees that all sustainability credit transfers are atomic and durable.

- This ensures that the campus "Sustainability Ledger" remains accurate and resistant to system failures or concurrent data entry errors.